

14 — Development Preparation

Purpose

The purpose of this document is to define the preparation process required before beginning Mosaica implementation.

This module transforms the approved architecture into a practical development environment.

The objective is to ensure that development begins with:

- appropriate tools,
 - clear technical decisions,
 - organized project structure,
 - controlled implementation steps.
-

Development Preparation Scope

This module defines:

- technology decisions,
 - development environment,
 - repository setup,
 - initial project structure,
 - development workflow,
 - first implementation strategy.
-

Development Preparation Position

Development Preparation acts as the bridge between architecture and implementation.

Approved Architecture

↓

Development Preparation

↓

Implementation

Preparation Principles

Principle 1 — Simple Start

The initial development environment should be as simple as possible.

Avoid unnecessary tools and dependencies.

Principle 2 — Free and Accessible Tools First

Development decisions should prioritize:

- free tools,
- local development,
- low-cost solutions.

Paid services should only be introduced when justified.

Principle 3 — Architecture Before Code

Implementation should follow approved architecture.

Code should not create architecture accidentally.

Principle 4 — Incremental Development

Development should proceed through small validated steps.

Principle 5 — Tool Choice Supports Product Goals

Technology decisions should serve Mosaica's needs rather than follow trends.

Development Preparation Model

Technology

↓

Environment

↓

Repository

↓

Structure

↓

Workflow

↓

Implementation

Module Structure

14 — Development Preparation

14.01 Technology Selection

14.02 Development Environment Setup

14.03 Repository Initialization

14.04 Initial Project Structure

14.05 Development Workflow

14.06 First Implementation Plan

14.07 Development Preparation Review

14.01 — Technology Selection

Purpose

The purpose of this section is to define the technology stack selected for Mosaica development.

This decision establishes the technical foundation required to implement the approved architecture.

The objective is to select technologies that balance:

- simplicity,
- maintainability,
- learning curve,
- future scalability,
- cost efficiency.

Technology Selection Principles

Principle 1 — Choose Based on Product Needs

Definition

Technology decisions shall serve Mosaica's requirements rather than following trends.

Principle 2 — Prefer Accessible Technologies

Definition

The selected technologies should be widely available and supported.

Principle 3 — Avoid Premature Complexity

Definition

The initial technology stack should contain only what is required.

Principle 4 — Preserve Future Flexibility

Definition

Initial choices should allow future expansion without forcing immediate complexity.

Evaluated Technology Approaches

Option 1 — Full JavaScript Stack

Example:

Frontend:
React

Backend:
Node.js

Database:
PostgreSQL

Advantages

- Large ecosystem
- Many learning resources
- Popular industry approach

Disadvantages

- Multiple concepts to learn
 - Frontend and backend separation increases complexity
-

Option 2 — Python Full Stack

Example:

Backend:
Django

Database:
SQLite/PostgreSQL

Frontend:
Django Templates

Advantages

- Fast development
- Simple starting point
- Good learning experience

Disadvantages

- Less flexible for highly interactive modern interfaces

Option 3 — Modern Full Stack Framework

Example:

Frontend:

Next.js

Backend:

Next.js API

Database:

PostgreSQL

Advantages

- Modern architecture
- Single application structure
- Good future scalability

Disadvantages

- Slightly higher initial learning curve

Recommended Technology Stack

After evaluating Mosaica's requirements:

The recommended initial stack is:

Frontend:

Next.js

Backend:

Next.js API Routes

Database:

PostgreSQL

ORM:

Prisma

Language:

TypeScript

Why This Choice?

1. Single Project Structure

Instead of:

Frontend Project

+

Backend Project

we begin with:

Mosaica

├─ Frontend

├─ API

└─ Database Connection

This reduces management complexity.

2. Future Growth Compatibility

This structure can support:

- web application,
 - mobile API,
 - authentication,
 - integrations,
 - future AI features.
-

3. Strong Development Ecosystem

Next.js and TypeScript provide:

- documentation,
 - community support,
 - long-term relevance.
-

4. Cost Compatibility

Initial development requires:

- no server purchase,
- no paid database,
- no AI subscription.

Local development is possible.

Database Decision

Initial database approach:

Development:

SQLite / Local Database

Future Production:

PostgreSQL

Burada küçük ama önemli bir düzeltme yapıyorum.

Mimari olarak PostgreSQL belirledik ama başlangıçta senin maliyet hassasiyetin nedeniyle:

ilk prototipte PostgreSQL kurmak zorunda değiliz.

Başlangıç:

Bilgisayarında çalışan Mosaica

↓

SQLite

olabilir.

Daha sonra:

Kullanıcı artışı veya yayın aşaması

↓

PostgreSQL

geçişi yapılabilir.

Initial Technology Stack Decision

Mosaica v0.x

Framework:
Next.js

Language:
TypeScript

Database:
SQLite

ORM:
Prisma

Version Control:
Git + GitHub

Editor:
Visual Studio Code

Excluded Initial Technologies

The following are intentionally not required:

- ✗ AI APIs
- ✗ Cloud servers
- ✗ Paid databases
- ✗ Complex DevOps systems
- ✗ Enterprise infrastructure

Technology Evolution Path

Local Development

↓

Personal Usage

↓

Optional Deployment

↓

Production System

Architectural Decision

Decision

Mosaica will initially use a modern TypeScript-based full-stack architecture with local-first development.

Rationale

This approach provides the best balance between:

- development simplicity,
- future scalability,
- cost control,
- maintainability.

14.02 — Development Environment Setup

Purpose

The purpose of this section is to define the development environment required for Mosaica implementation.

This section establishes the minimum tools needed to develop, test and maintain the application.

The objective is to create a simple, reliable and cost-effective development environment.

Environment Principles

Principle 1 — Minimum Required Tools

Definition

Only necessary development tools should be installed.

Application

The initial environment should avoid:

- unnecessary software,
 - complex configurations,
 - unused services.
-

Principle 2 — Free Development Environment

Definition

The initial development environment should use free tools whenever possible.

Required Tools

The initial Mosaica development environment requires:

Visual Studio Code

Node.js

Git

GitHub Account

Web Browser

1. Visual Studio Code

Purpose

Primary code editor for Mosaica development.

Responsibilities

Used for:

- writing code,
 - managing project files,
 - running development commands,
 - extensions.
-

Cost

Free.

2. Node.js

Purpose

Provides the runtime environment required for Next.js development.

Responsibilities

Used for:

- running the application,
 - installing packages,
 - executing development commands.
-

Cost

Free.

3. Git

Purpose

Version control system.

Responsibilities

Used for:

- tracking changes,
 - saving development history,
 - managing versions.
-

Cost

Free.

4. GitHub Account

Purpose

Remote repository management.

Responsibilities

Used for:

- storing project history,
 - backup,
 - future collaboration.
-

Cost

Free plan is sufficient initially.

5. Web Browser

Purpose

Testing application behavior.

Recommended

Any modern browser can be used.

Examples:

- Chrome,
 - Edge,
 - Firefox.
-

Optional Tools

The following tools are not required initially:

Docker

Cloud Platforms

Database Hosting

AI Development Tools

Advanced CI/CD Services

Local Development Model

Initial development follows:

Computer

↓

Development Environment

↓

Local Application

↓

Local Database

↓

Testing

Environment Structure

The initial setup will contain:

Berke's Computer

├─ VS Code

├─ Node.js

├─ Git

└─ Mosaica Project Folder

Installation Order

The recommended installation order:

1. Install VS Code

↓

2. Install Node.js

↓

3. Install Git

↓

4. Create GitHub Account

↓

5. Create Mosaica Project

Environment Security

The development environment should follow basic security principles.

Rules:

- Do not store passwords in code.
 - Do not upload private keys.
 - Keep credentials separate.
-

Future Environment Expansion

Additional tools may be introduced when needed.

Examples:

- Docker,
- database tools,
- deployment tools,
- testing tools.

They should only be added when justified.

Development Environment Decision

Decision

Mosaica development will begin with a lightweight local development environment using free tools.

Rationale

This approach provides:

- zero initial cost,
- simple setup,
- full development capability,
- future expansion flexibility.

14.03 — Repository Initialization

Purpose

The purpose of this section is to define the initial repository setup for Mosaica development.

This section establishes how the project source code will be stored, organized and version-controlled.

The objective is to create a reliable foundation for development history and future collaboration.

Repository Principles

Principle 1 — Single Source of Truth

Definition

The repository shall contain the authoritative version of the Mosaica source code.

Application

The latest approved code state should always exist in the repository.

Principle 2 — Track Development History

Definition

Changes should be recorded through version control.

Application

Git history should explain:

- what changed,
 - when it changed,
 - why it changed.
-

Principle 3 — Protect Project Structure

Definition

Repository organization should remain clean and understandable.

Application

Avoid:

- unnecessary files,
 - temporary experiments,
 - unrelated assets.
-

Repository Platform Decision

Selected Platform

GitHub

Reason

GitHub provides:

- free repository hosting,
 - Git integration,
 - backup,
 - future collaboration support.
-

Repository Naming

Recommended repository name:

Naming Principles

The repository name should be:

- simple,
- lowercase,
- memorable,
- consistent.

Repository Visibility

Initial recommendation:

Private Repository

Reason

At this stage:

- development is personal,
- code is experimental,
- project decisions are still evolving.

Public release can be considered later.

Initial Repository Structure

The repository begins with:

mosaica/

|— app/

|— components/

|— lib/

|— prisma/

|— public/

```
|— docs/
|— README.md
|— package.json
└— .gitignore
```

Bu yapının detayını bir sonraki bölümde (**14.04 Initial Project Structure**) açacağız.

Şu an sadece repository seviyesinde tanımlıyoruz.

Initial Git Strategy

Development starts with:

main branch

↓

Development changes

↓

Commit history

Initial Commit

The first commit should represent:

Initial Mosaica Project Setup

Purpose

Creates the first stable reference point.

Commit Principles

Commits should be:

- small,
- understandable,
- meaningful.

Example

Good:

Add user library model

Bad:

Changes

Repository Backup Strategy

GitHub provides remote backup for:

- source code,
 - configuration,
 - project history.
-

Files Not Stored

The repository should not contain:

Passwords

API Keys

Private Credentials

Temporary Files

Large Generated Files

Future Repository Expansion

Future additions may include:

Documentation

Testing

Deployment Configuration

Automation

when needed.

Repository Decision

Decision

Mosaica will use a private GitHub repository named `mosaica` for source control and development history.

Rationale

This provides:

- free backup,
- controlled development,
- clear version tracking,
- future scalability.

14.04 — Initial Project Structure

Purpose

The purpose of this section is to define the initial file and folder structure of the Mosaica project.

This structure establishes how application code, data models, documentation and resources are organized.

The objective is to create a maintainable project foundation aligned with the approved architecture.

Project Structure Principles

Principle 1 — Structure Follows Responsibility

Definition

Files and folders should be organized according to their responsibility.

Application

The project structure should reflect:

Feature Responsibility

↓

Code Location

↓

Clear Ownership

Principle 2 — Avoid Premature Complexity

Definition

The initial structure should contain only necessary folders.

Application

Avoid creating empty structures for future possibilities.

Principle 3 — Support Future Growth

Definition

The structure should allow expansion without requiring complete reorganization.

Initial Project Structure

The initial Mosaica project structure:

```
mosaica/
```

```
├─ app/  
├─ components/  
├─ lib/  
├─ prisma/  
├─ public/  
├─ docs/  
├─ tests/  
├─ .env.example  
├─ .gitignore  
├─ package.json  
├─ README.md  
└─ tsconfig.json
```

Folder Responsibilities

/app

Purpose

Main application layer.

Contains:

- pages,
 - routes,
 - application entry points.
-

Architecture Relationship

User Interface

↓

Application Entry

/components

Purpose

Reusable interface components.

Examples:

- buttons,
 - cards,
 - navigation elements,
 - content displays.
-

Architecture Relationship

Supports:

UX Architecture

↓

Interface Implementation

/lib

Purpose

Shared application logic and utilities.

Contains:

- helper functions,
 - configuration,
 - shared services.
-

Architecture Relationship

Supports:

Application Layer

↓

Shared Logic

/prisma

Purpose

Database-related definitions.

Contains:

- database schema,
 - migrations,
 - ORM configuration.
-

Architecture Relationship

Connects:

Domain Model

↓

Database Implementation

/public

Purpose

Static assets.

Contains:

- images,
 - icons,
 - public files.
-

/docs

Purpose

Project documentation.

Contains:

- architecture documents,
 - decisions,
 - development notes.
-

Importance

Keeps project knowledge together with the code.

/tests

Purpose

Testing files.

Contains:

- automated tests,
 - validation scenarios.
-

Configuration Files

package.json

Purpose

Defines:

- project dependencies,

- scripts,
 - metadata.
-

tsconfig.json

Purpose

TypeScript configuration.

.gitignore

Purpose

Defines files Git should not track.

Examples:

```
node_modules/
```

```
.env
```

```
temporary files
```

.env.example

Purpose

Documents required environment variables without exposing secrets.

Example:

```
DATABASE_URL=
```

README.md

Purpose

Main project documentation.

Contains:

- project description,
- setup instructions,
- development notes.

Architecture Mapping

The project structure maps to architecture:

Mosaica Architecture

UX
↓
/app
↓
/components

↓
Application Logic
↓
/lib

↓
Database Layer
↓
/prisma

Initial Structure Decision

Decision

Mosaica will begin with a feature-ready but simple project structure based on Next.js conventions.

Rationale

This structure provides:

- clear organization,
- easy navigation,
- future scalability,
- minimal initial complexity.

Excluded Initial Folders

The following are intentionally not created yet:

/cloud

/ai

/mobile

/admin

/analytics

Reason

These represent future capabilities, not initial requirements.

14.05 — Development Workflow

Purpose

The purpose of this section is to define the development workflow used during Mosaica implementation.

This workflow establishes how changes are planned, developed, tested and integrated into the project.

The objective is to create a consistent development process that supports:

- quality,
- organization,

- maintainability,
 - controlled progress.
-

Development Workflow Principles

Principle 1 — Plan Before Implementation

Definition

Development tasks should begin with understanding the goal before writing code.

Application

Before implementing a feature:

Requirement

↓

Design Decision

↓

Implementation

↓

Validation

Principle 2 — Small Incremental Progress

Definition

Development should proceed through small, understandable steps.

Application

Prefer:

Small Feature

↓

Test

↓

Improve

Instead of:

Large System Change

↓

Unknown Problems

Principle 3 — Keep Main Branch Stable

Definition

The main branch should represent a working version of Mosaica.

Application

Development changes should be tested before becoming the main version.

Branch Strategy

Initial strategy:

main

↓

feature branches

↓

merge to main

Main Branch

Purpose

Contains stable project versions.

Rule

The main branch should always remain usable.

Feature Branches

Purpose

Used for developing specific changes.

Naming Example

`feature/library-system`

`feature/content-page`

`feature/user-profile`

Development Cycle

Each feature follows:

Feature Idea

↓

Create Task

↓

Create Branch

↓

Develop

↓

Test

↓

Review

↓

Merge

Commit Strategy

Commits should represent meaningful progress.

Good Commit Examples

Create initial database schema

Add content entity model

Implement library page

Avoid

Update files

Changes

Fix stuff

Testing Workflow

Before accepting a change:

Check:

Does it work?

↓

Does it break existing features?

↓

Does it follow architecture?

Error Management

Errors should be handled through:

Problem

↓

Investigation

↓

Solution

↓

Documentation if needed

Development Documentation

Important decisions should be recorded.

Examples:

- architecture changes,
 - technology changes,
 - major feature decisions.
-

Development Tools Usage

Initial workflow tools:

VS Code

+

Git

+

GitHub

+

Browser Testing

AI Tool Usage

AI assistance may be used as a development helper.

However:

AI output must be reviewed.

AI does not replace:

- architectural decisions,
- testing,
- developer understanding.

Development Workflow Model

Idea

↓

Decision

↓

Code

↓

Test

↓

Commit

↓

Version

Cost-Aware Development

Development workflow should avoid unnecessary expenses.

Preferred:

- local development,
 - free tools,
 - open-source solutions.
-

Development Workflow Decision

Decision

Mosaica will use an incremental Git-based development workflow focused on small validated improvements.

Rationale

This approach provides:

- organized progress,
- easier debugging,
- safer development,
- maintainable code history.

14.06 — First Implementation Plan

Purpose

The purpose of this section is to define the first implementation plan for Mosaica development.

This section establishes the initial development sequence required to transform the approved architecture into a working application.

The objective is to create the first usable version through controlled and incremental implementation.

Implementation Principles

Principle 1 — Build the Foundation First

Definition

Initial development should establish the core system before adding advanced capabilities.

Application

The first implementation should focus on:

Project Setup

↓

Core Data Structure

↓

Basic User Experience

↓

Essential Features

Principle 2 — Validate Early

Definition

A working system should be created as soon as possible.

Application

Avoid building large sections before testing the basic idea.

Principle 3 — Follow Architecture

Definition

Implementation should follow approved architectural decisions.

Application

Development order should respect:

Database

↓

Domain Logic

↓

Application Logic

↓

Interface

First Development Phase

Phase 1 — Project Initialization

Goal:

Create a working Mosaica development environment.

Tasks:

- create Next.js project,
 - configure TypeScript,
 - initialize Git,
 - connect repository.
-

Phase 2 — Database Foundation

Goal:

Create the first data foundation.

Tasks:

- configure Prisma,
- create initial database,
- define first models.

Initial focus:

Content

Library Entry

Collection

Phase 3 — Core Application Structure

Goal:

Create the first working application flow.

Tasks:

- create application routes,
 - connect data layer,
 - display stored information.
-

Phase 4 — First User Experience

Goal:

Create the first usable interaction.

Initial experience:

Open Mosaica

↓

View Library

↓

See Content

↓

Manage Personal Archive

Initial Feature Priority

The first implementation order:

1. Project Setup

↓

2. Content Model

↓

3. Library System

↓

4. Collection System

↓

5. Basic Interface

↓

6. Improvements

Features Not Included Initially

The first implementation will not include:

AI Features

Social Features

Advanced Analytics

External Integrations

Mobile Application

Payment Systems

First Working Version Goal

The first milestone is:

Mosaica v0.1

Definition:

A local application where the User can:

- create content entries,
 - view personal library,
 - organize items,
 - store basic personal information.
-

Implementation Milestones

Milestone 1 — Running Application

Success criteria:

Mosaica opens locally

Milestone 2 — Data System

Success criteria:

Content can be stored and retrieved

Milestone 3 — Personal Archive

Success criteria:

User can manage Library entries

Milestone 4 — Usable Experience

Success criteria:

Core workflow works from start to finish

Development Starting Point

The first practical development action will be:

Install Required Tools

↓

Create Next.js Project

↓

Run First Local Version

First Implementation Decision

Decision

Mosaica development will begin with a local-first Minimum Viable Product approach.

Rationale

This approach:

- reduces risk,
- avoids unnecessary costs,
- validates the core idea quickly,
- creates a foundation for future expansion.

14.07 — Development Preparation Review

Purpose

The purpose of this review is to validate that the Mosaica development preparation phase has created a sufficient foundation for implementation.

This review confirms that:

- technology decisions are defined,
- development tools are prepared,
- repository structure is established,
- implementation steps are clear.

Review Scope

The following preparation areas are reviewed:

14.01 Technology Selection

14.02 Development Environment Setup

14.03 Repository Initialization

14.04 Initial Project Structure

14.05 Development Workflow

14.06 First Implementation Plan

Validation Results

Technology Selection

Status: Approved

The selected technology stack supports Mosaica's current needs.

Confirmed:

Next.js

TypeScript

Prisma

SQLite (initial)

Git + GitHub

The stack provides:

- low initial cost,
- manageable complexity,
- future growth capability.

Development Environment

Status: Approved

The required development environment is defined.

Confirmed:

Required tools:

Visual Studio Code

Node.js

Git

GitHub

Browser

No unnecessary tools are required.

Repository Setup

Status: Approved

The repository strategy is defined.

Confirmed:

Repository:

mosaica

Visibility:

Private

The repository will provide:

- backup,
- history,
- controlled development.

Project Structure

Status: Approved

The initial code organization is defined.

Confirmed:

app

components

lib

prisma

public

docs

tests

The structure supports the approved architecture.

Development Workflow

Status: Approved

The development process is defined.

Confirmed:

Plan

↓

Develop

↓

Test

↓

Commit

↓

Improve

Implementation Plan

Status: Approved

The first development phase is clearly defined.

Initial goal:

Mosaica v0.1

Capabilities:

- local application,
 - content management,
 - personal library,
 - basic organization.
-

Cost Validation

Status: Approved

The development plan respects initial financial constraints.

Confirmed:

The project can begin without:

- paid hosting,
- paid database,
- AI API,
- commercial tools.

Scope Validation

Status: Approved

The initial scope remains controlled.

Not included:

AI

Mobile App

Social System

Payments

Advanced Analytics

These remain future possibilities.

Development Preparation Conclusion

The Development Preparation phase is complete.

Mosaica has:

- a defined technology foundation,
- a prepared development environment,
- a repository strategy,
- a project structure,
- a first implementation plan.

The project is ready to transition into implementation.
